

# C Language — Cheatsheet com Exercícios (pythontutor.com)

## Exercícios:

<https://claude.ai/public/artifacts/f99f6ab2-60aa-4d8e-8cf4-ddb24015076a>

## Site usado para realizá-los:

<https://pythontutor.com/c.html>



## Exercício 2 — Inspeccionando tamanhos

C

```
#include <stdio.h>
```

```
int main(void) {  
    printf("char:  %zu byte(s)\n", sizeof(char));  
    printf("int:   %zu byte(s)\n", sizeof(int));  
    printf("long:  %zu byte(s)\n", sizeof(long));  
    printf("float: %zu byte(s)\n", sizeof(float));  
    printf("double: %zu byte(s)\n", sizeof(double));  
    return 0;  
}
```

**O que observar:** os valores impressos confirmam a tabela acima.

Valores impressos com o código acima:

```
char:  1 byte(s)  
int:   4 byte(s)  
long:  8 byte(s)  
float: 4 byte(s)  
double: 8 byte(s)
```

Tente mudar `sizeof(int)` para `sizeof(short)` e compare.

```
char:  1 byte(s)  
int:   2 byte(s)  
long:  8 byte(s)  
float: 4 byte(s)  
double: 8 byte(s)
```

O modificador de tamanho (short), muda o número de bytes (espaço) que uma variável ocupa na memória, encolhendo-o para economizar memória (cortou o int pela metade).



### Exercício 3 — Variável não inicializada vs. inicializada

C

```
#include <stdio.h>
```

```
int main(void) {  
    int a = 42;  
    int b;      /* valor lixo — NÃO leia antes de atribuir */  
    b = a + 8;  
    printf("a = %d\n", a);  
    printf("b = %d\n", b);  
  
    const int N = 100;  
    /* N = 200; */ /* descomente para ver o erro de compilacao */  
    printf("N = %d\n", N);  
    return 0;  
}
```

Valor impresso do código acima:

Print output (drag lower right corner to resize)

```
a = 42  
b = 50  
N = 100
```

|      | Stack      | Heap |
|------|------------|------|
| main |            |      |
| a    | int<br>42  |      |
| b    | int<br>50  |      |
| N    | int<br>100 |      |

**O que observar:** o valor de **a** aparece imediatamente no frame; **b** só recebe valor após a atribuição.

Tente descomentar **N = 200** e veja o erro.

error: assignment of read-only variable 'N'

Isso acontece quando se usa a palavra-chave `const` antes do tipo de uma variável, isso torna aquele valor constante (imutável), sendo assim inalterável após a sua inicialização.



## Exercício 4 — Divisão inteira, módulo e incremento

C

```
#include <stdio.h>
```

```
int main(void) {  
    int a = 17, b = 5;  
  
    printf("%d / %d = %d\n", a, b, a / b); /* divisao inteira */  
    printf("%d %% %d = %d\n", a, b, a % b); /* resto */  
  
    int x = 3;  
    int pre = ++x; /* x vira 4, pre = 4 */  
    int pos = x++; /* pos = 4, x vira 5 */  
    printf("pre=%d pos=%d x=%d\n", pre, pos, x);  
  
    int max = (a > b) ? a : b;  
    printf("max(%d,%d) = %d\n", a, b, max);  
    return 0;  
}
```

**O que observar:** diferença entre pré e pós-incremento no valor retornado; resultado da divisão inteira vs. real.

Retorna:

Print output (drag lower right corner to resize)

```
17 / 5 = 3
17 % 5 = 2
pre=4 pos=4 x=5
max(17,5) = 17
```

Stack

Heap

| main |           |
|------|-----------|
| a    | int<br>17 |
| b    | int<br>5  |
| x    | int<br>5  |
| pre  | int<br>4  |
| pos  | int<br>4  |
| max  | int<br>17 |

**Pré-incremento (++x):** Primeiro o código **soma 1** à variável, e depois entrega o valor atualizado para a expressão.

**Pós-incremento (x++):** Primeiro o código **entrega o valor atual** da variável para a expressão, e só depois soma 1 a ela.



### Exercício 5a — if / else if / else

c

```
#include <stdio.h>
```

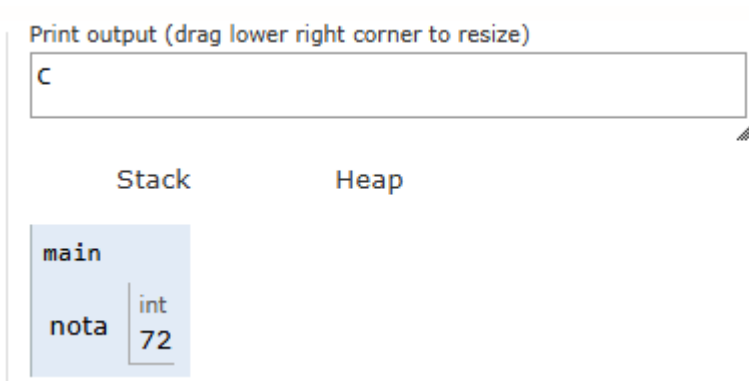
```
int main(void) {
    int nota = 72;

    if (nota >= 90) printf("A\n");
    else if (nota >= 80) printf("B\n");
    else if (nota >= 70) printf("C\n");
    else printf("Reprovado\n");

    return 0;
}
```

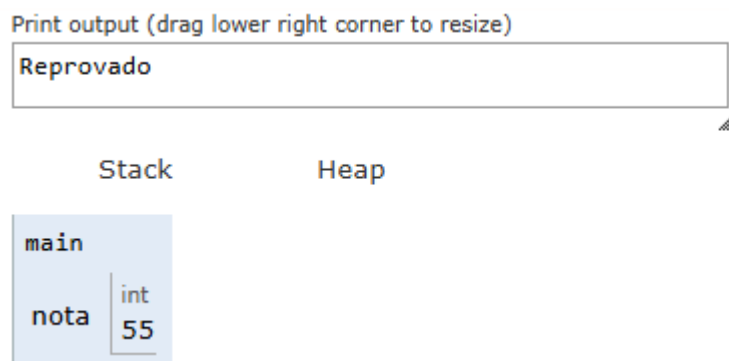
Valores retornados:

Código sem alterações



**Variação:** troque **nota** por 55, 80 e 95; execute cada versão e acompanhe qual ramo é percorrido.

Código com a nota = 55



Código com a nota = 80



Código com a nota = 90



### Exercício 5b — for com break e continue

C

```
#include <stdio.h>
```

```
int main(void) {
    for (int i = 0; i < 10; i++) {
        if (i % 2 == 0) continue; /* pula pares */
        if (i == 7) break; /* para no 7 */
        printf("%d\n", i);
    }
    return 0;
}
```

**O que observar:** o valor de `i` no frame a cada passo; quando `continue` pula a impressão e quando `break` encerra o laço.

O laço for testa a variável `i` de 0 a 9. Quando `i` é par (0, 2, 4, 6, ou seja, o resto da divisão por 2 é igual a zero), o comando `continue` ignora o `printf` e pula para o próximo número. Quando `i` chega a 7, o `break` encerra o laço imediatamente, impedindo que os números 7, 8 e 9 sejam processados. Apenas 1, 3 e 5 (ímpares) são impressos.

### Exercício 5c — do-while

C

```
#include <stdio.h>
```

```
int main(void) {
    int n = 1;
    do {
        printf("n = %d\n", n);
        n *= 2;
    } while (n < 32);
    return 0;
}
```

**O que observar:** o corpo executa *antes* do teste; **n** dobra a cada iteração (potências de 2).

O que o código abaixo mostra quando executado:

```
n = 1
n = 2
n = 4
n = 8
n = 16
```

Stack

Heap

```
main
├── int
│   └── 32
└── n
```

O corpo dobra o **n** antes do teste, pois o teste apenas determina se a variável atende aos critérios da condição estabelecida no final do loop. Caso seja afirmativo, o loop roda mais uma vez, como não foi, a variável permaneceu com o valor dobrado, sem ser impresso na tela.

### Exercício 5d — switch com fall-through

c

```
#include <stdio.h>
```

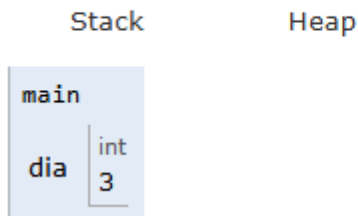
```
int main(void) {
    int dia = 3; /* 1=Seg ... 7=Dom */

    switch (dia) {
        case 1: case 2: case 3: case 4: case 5:
            printf("Dia util\n");
            break;
        case 6: case 7:
            printf("Fim de semana\n");
            break;
        default:
            printf("Invalido\n");
    }
    return 0;
}
```

**O que observar:** o *fall-through* intencional agrupa casos; remova um **break** para ver o comportamento inesperado.

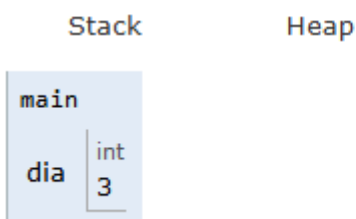
Saída com o break:

```
Dia util
```



Saída sem o primeiro break:

```
Dia util
Fim de semana
```



Quando o programa encontra um case que corresponde ao valor da variável, ele entra ali e começa a executar o código. A partir desse ponto, o computador ignora todos os outros case seguintes e continua executando linha por linha, de cima para baixo, até que encontre um comando break ou o switch termine. Como o dia era 3 e se enquadrou no primeiro grupo, ele foi considerado válido, sem o break, ele executou linha por linha até o break mais próximo.

## Exercício 6 — Pilha de chamadas em ação

c

```
#include <stdio.h>
```

```
int fatorial(int n) {
    if (n <= 1) return 1;
    return n * fatorial(n - 1); /* chamada recursiva */
}
```

```
void dobra(int *x) { *x *= 2; }
```

```
int main(void) {
```



```

int f = fatorial(5);
printf("5! = %d\n", f);

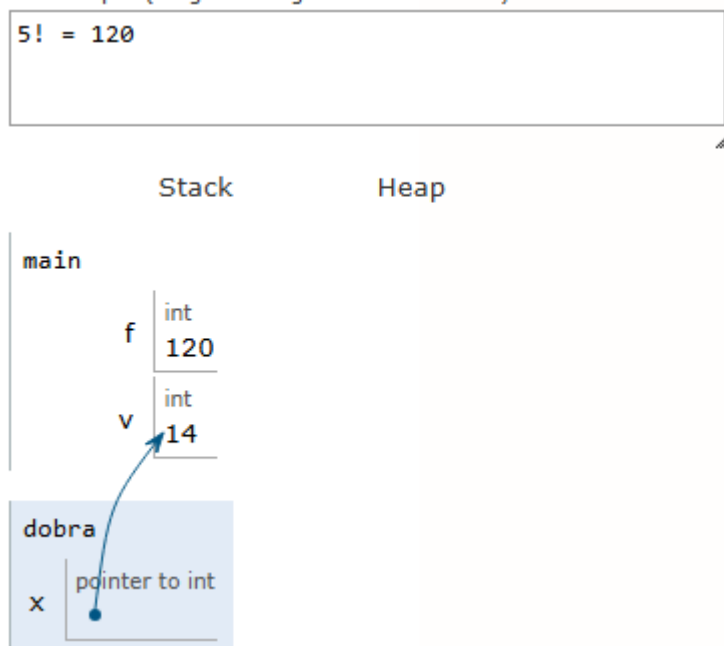
int v = 7;
dobra(&v);
printf("dobro de 7 = %d\n", v);
return 0;
}

```

**O que observar:** os frames de `fatorial` empilhando e desempilhando; como `dobra` modifica `v` via ponteiro — o frame de `dobra` mostra `x` apontando para `v` em `main`.

Na primeira parte, a função `fatorial` realiza chamadas recursivas, o que faz com que o sistema empilhe múltiplos frames na memória (de `fatorial(5)` até `fatorial(1)`), onde cada um aguarda o resultado do próximo. Ao atingir o caso base ( $n \leq 1$ ), a pilha é desempilhada de trás para frente, resolvendo as multiplicações consecutivas até retornar o valor final de 120 para a função `main`.

Na segunda parte, a função `dobra` utiliza ponteiros para modificar o valor de uma variável local da `main`. Ao passar o endereço de memória (`&v`), o frame da função `dobra` cria um ponteiro `x` que aponta diretamente para o endereço de `v` no frame da `main`. A operação de desreferenciação (`*x *= 2`) altera o valor diretamente na origem, fazendo com que a variável `v` passe a valer 14 de forma permanente.



Saída final:

```
5! = 120
dobro de 7 = 14
```

Stack

Heap

| main |            |
|------|------------|
| f    | int<br>120 |
| v    | int<br>14  |

## Exercício 7 — Array unidimensional e matriz

c

```
#include <stdio.h>
```

```
int main(void) {
    int v[5] = {10, 20, 30, 40, 50};

    /* soma dos elementos */
    int soma = 0;
    for (int i = 0; i < 5; i++)
        soma += v[i];
    printf("soma = %d\n", soma);

    /* matriz 2x3 */
    int m[2][3] = {{1, 2, 3}, {4, 5, 6}};
    printf("m[1][2] = %d\n", m[1][2]); /* 6 */
    return 0;
}
```

**O que observar:** o PythonTutor mostra o array como células numeradas; acompanhe **soma** acumulando a cada iteração.

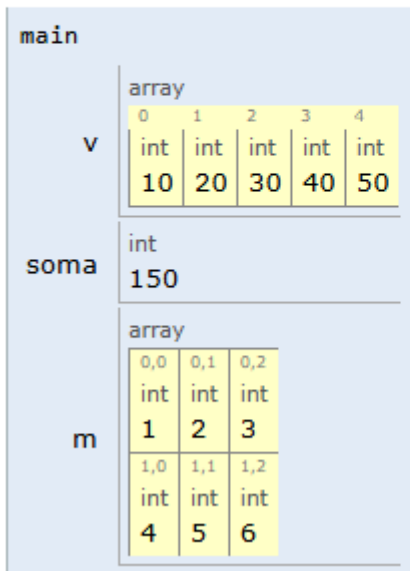
O vetor de 5 elementos é percorrido por um laço for, onde a variável soma armazena as somas de seu próprio valor + o valor de v[i] ( soma += v[i]). Depois, é declarada uma matriz de duas linhas e três colunas, com os valores de 1 a 6. Por fim, é impresso o valor contido na segunda linha e terceira coluna, que corresponde ao número 6.

Saída final:

```
soma = 150
m[1][2] = 6
```

Stack

Heap



## Exercício 8 — Percorrendo uma string caractere a caractere

c

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main(void) {
```

```
    char s[] = "Brasilia";
```

```
    int len = strlen(s);
```

```
    printf("Comprimento: %d\n", len);
```

```
    /* percorre e imprime cada char e seu codigo ASCII */
```

```
    for (int i = 0; i < len; i++)
```

```
        printf("s[%d] = '%c' (ASCII %d)\n", i, s[i], s[i]);
```

```
    /* comparacao */
```

```
    char t[] = "Brasilia";
```

```
    printf("strcmp: %d\n", strcmp(s, t)); /* 0 = iguais */
```

```
    return 0;
```

```
}
```

**O que observar:** o array `s` no frame de `main` com cada byte; o `'\0'` invisível no último índice (valor 0). Compare com `char *s = "Brasilia"` — o PythonTutor mostra `p` como uma seta para fora do frame (memória estática).

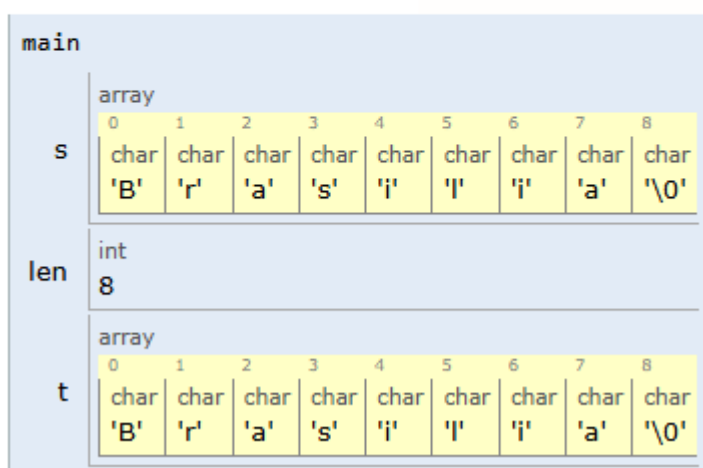
O código demonstra a manipulação de cadeias de caracteres (strings) na linguagem C. a string `s`, com o texto "Brasilia", e armazenada na memória como um vetor do tipo `char`. Este tipo de vetor finaliza a palavra com o caractere nulo (`\0`).

A variável `len` armazena o comprimento do texto retornado pela função `strlen(s)`. Depois, um laço `for` percorre esse vetor índice por índice, onde o `printf` exibe a letra literal (`%c`) e o seu respectivo código numérico decimal na tabela ASCII (`%d`). Por fim, é declarada uma segunda string `t` idêntica e o programa imprime o resultado da função `strcmp(s, t)`, que corresponde ao número 0 porque as duas strings são iguais.

Saída do código sem alteração:

```
s[4] = 'i' (ASCII 105)
s[5] = 'l' (ASCII 108)
s[6] = 'i' (ASCII 105)
s[7] = 'a' (ASCII 97)
strcmp: 0
```

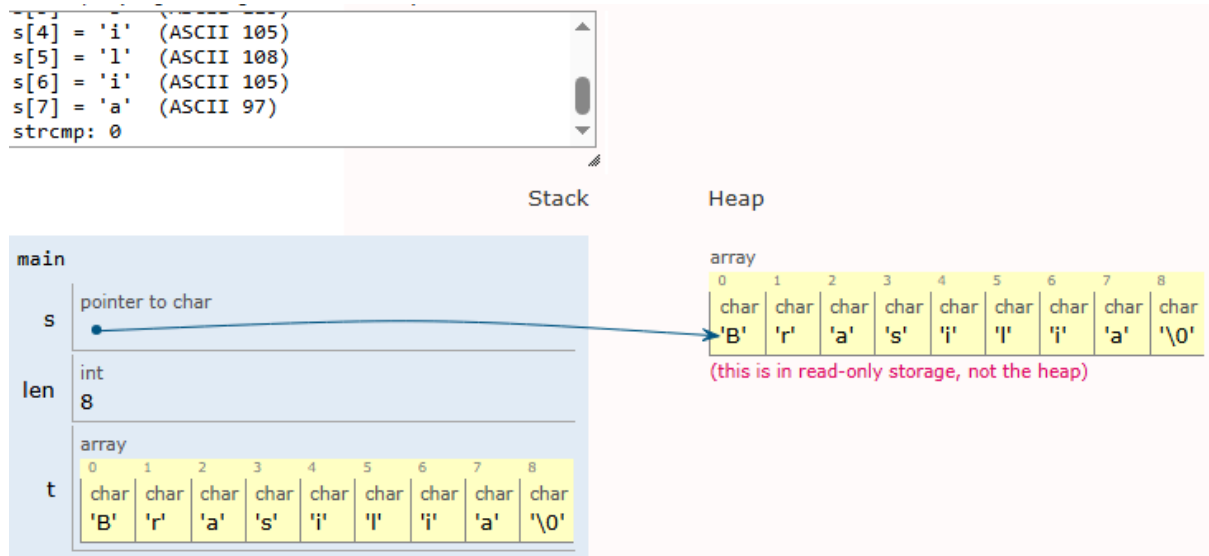
Stack



A diferença está em como a string é guardada na memória. No código anterior (`char s[]`), "Brasilia" é um vetor comum salvo dentro da função `main`, o que permite alterar as letras se necessário. Já no novo código (`char *p`), a palavra fica salva em uma área especial da memória que é de apenas leitura. A variável `p` é um ponteiro (uma seta) que apenas guarda o

endereço de onde essa palavra está. Por isso, qualquer tentativa de mudar alguma letra usando `p` fará o programa dar erro e fechar sozinho.

Saída do código com `char *s = "Brasilia":`



## Exercício 9a — Ponteiro básico e de referência

c

```
#include <stdio.h>
```

```
int main(void) {
    int x = 10;
    int *p = &x;

    printf("x  = %d\n", x);
    printf("&x = %p\n", (void*)&x);
    printf("p  = %p\n", (void*)p);
    printf("*p = %d\n", *p);

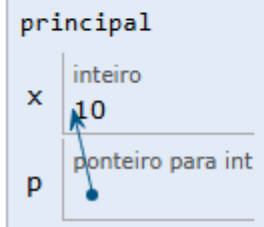
    *p = 99;
    printf("x apos *p=99: %d\n", x);
    return 0;
}
```

**O que observar:** no PythonTutor, `p` é exibido como uma **seta** apontando para a caixa de `x`. Após `*p = 99`, a caixa de `x` muda de valor sem que `x` apareça no lado esquerdo da atribuição.

```
x    = 10
&x   = 0xffff000bd4
p    = 0xffff000bd4
```

Pilha

Monte



```
x    = 10
&x   = 0xffff000bd4
p    = 0xffff000bd4
*p   = 10
x apos *p=99: 99
```

Pilha

Monte

